Similar to `explode()` is the `str_split()` function:

```
var_export(str_split("Thanks for all the fish."));
```

```
Output:
array (
  0 => 'T',
  1 => 'h',
  2 => 'a',
```

By default `str_split()` returns an array of single bytes but will accept a second integer argument to control the byte length of each element in the array:

```
var_export(str_split("Thanks for all the fish.", 12));
```

```
Output:
array (
  0 => 'Thanks for a',
  1 => 'll the fish.',
)
```

Naturally, this will present problems with multi-byte encodings:

```
$bytes = str_split("\u{5408}\u{6c23}\u{9053}"); // 合氣道 - will be UTF-8
                                                   encoded on *nix systems
foreach ($bytes as $byte) {
    print "0x" . bin2hex($byte) . " ";
}
```

```
Output:
0xe5 0x90 0x88 ...
```

The three Unicode escape sequences generate a UTF-8 encoded string, but `str_split()` is not encoding-aware and simply chops it into an array of single bytes. We can see the byte values in the output thanks to `bin2hex()`, and the first three are indeed the correct encoding for the 合 character. Not particularly useful, the original meaning of the text has been thoroughly mangled and would require careful reassembly to restore. Fortunately there is an `mbstring` equivalent function, `mb_str_split()`, which works on characters rather than bytes: